CNN Chest X-Ray Pneumonia Classifier: Project Report

Oct 13, 2026 · @Sam

Summary

A convolutional neural network (CNN) trained from scratch separated NORMAL from PNEUMONIA chest X-rays with 0.908 test accuracy and 0.867 recall on the NORMAL class, but only on synthetic demo data, so these figures do not describe real X-rays.

The project handles the three data problems the exercise is built around: images of different sizes and colour modes, a 3:1 class imbalance, and a validation folder of only 16 images. It converts every image to 150×150 grayscale, builds its own stratified 85/15 train/validation split, and passes balanced class weights into training.

On the synthetic test set the model made 5 false negatives and 6 false positives. A useless model that always answers PNEUMONIA would score 0.625 accuracy there, so per-class recall is reported alongside accuracy.

Caveat: the Kaggle dataset could not be downloaded in the environment where this was built. Every number below comes from a generated stand-in with the same folder layout and the same quirks. Re-running on the real data (instructions in the last section) is required before drawing any conclusion about pneumonia detection.

Introduction and objectives

The goal was to build a binary image classifier for NORMAL versus PNEUMONIA chest X-rays and to evaluate it honestly when the classes are imbalanced. The exercise looks simple, with only two classes, but deliberately uses a dataset with realistic preprocessing problems.

Four learning objectives guided the work:

- Build a preprocessing pipeline that copes with inconsistent size and colour mode.
- Diagnose and correct for class imbalance.
- Build and train a CNN from scratch, without pretrained weights.
- Evaluate with per-class metrics, because accuracy alone misleads under imbalance.

The work follows six tasks (inspect, split, preprocess, build, train, evaluate) plus two optional bonus experiments: an ablation without class weights, and a frozen MobileNetV2 transfer-learning model.

Data

The intended dataset is the Kaggle *Chest X-Ray Images (Pneumonia)* collection, about 1.2 GB, with `train/`, `test/` and `val/` folders each split into NORMAL and PNEUMONIA. Because it could not be downloaded here, the numbers in this report come from a synthetic stand-in generated with the same layout, a roughly 3:1 imbalance, varying image sizes, mixed grayscale and RGB files, and a 16-image `val/` folder. The synthetic data is not medical data.

Inspecting the raw synthetic training data confirmed all three problems the exercise describes:

Finding	Value
Training images, NORMAL	110
Training images, PNEUMONIA	330
Imbalance ratio (PNEUMONIA / NORMAL)	3.00×
Test images, NORMAL / PNEUMONIA	45 / 75
Distinct image sizes in `train/`	99
Colour modes in `train/`	290 grayscale, 150 RGB
Provided `val/` folder	16 images, ignored

The size and colour inconsistency means no image can be batched until it is resized and converted. The imbalance means a plain accuracy score would be dominated by the PNEUMONIA class. Sixteen validation images is too few to trust: a single image shifts any metric by more than 6 percentage points.

Method

Each design choice answers one of the data problems, and the order of the preprocessing steps matters.

Step	What was done	Why
Split	`train/` split 85/15, stratified by class (374 train, 66 validation); `val/` ignored	Sixteen images cannot support decisions such as early stopping; stratifying keeps the class ratio in both parts
Colour mode	Every image converted to grayscale	X-rays carry no colour signal; one channel instead of three cuts compute
Resize	150×150 pixels	Batching needs a constant tensor shape
Normalize	Pixel values scaled to [0, 1]	Keeps gradients well behaved
Augment	Training set only: 50% horizontal flip, rotation ±15°, zoom 0.9×–1.1×, brightness 0.9×–1.1×	Adds variety against overfitting; augmenting validation or test data would make their scores meaningless
Class weights	Balanced weights from the original training labels: NORMAL 2.01, PNEUMONIA 0.67	A NORMAL mistake then costs about 3× a PNEUMONIA mistake; the weights come from original counts because augmentation does not change how common each class really is

Architecture. Three blocks of Conv2D, BatchNorm, ReLU and MaxPool with 32, 64 and 128 filters, then Flatten, Dropout (0.5), a Dense layer of 128 units and a single sigmoid output. The model has about 5.4 million parameters and uses no pretrained weights. BatchNorm uses a momentum of 0.9 so its running statistics settle quickly on a small dataset; an early run with the default value collapsed to a constant prediction.

Training. Adam optimizer, binary cross-entropy loss, batch size 32, class weights passed to `model.fit`, and early stopping on validation loss (patience 4, best weights restored) with a ceiling of 25 epochs. The test set was used once, after training.

Results

On the untouched synthetic test set of 120 images the weighted CNN reached 0.908 accuracy, with recall of 0.867 on NORMAL and 0.933 on PNEUMONIA. These figures are from the executed notebook; a separate run of the command-line pipeline gave 0.883 accuracy and NORMAL recall of 0.822, so expect a few points of run-to-run variation on a test set this small.

Class	Precision	Recall	F1	Support
NORMAL	0.886	0.867	0.876	45
PNEUMONIA	0.921	0.933	0.927	75

Overall accuracy: 0.908. An always-PNEUMONIA model would score 0.625 on the same test set.

Confusion matrix (rows are the true class, columns the prediction):

	Predicted NORMAL	Predicted PNEUMONIA
Actual NORMAL	39	6
Actual PNEUMONIA	5	70

The recall on the NORMAL class, the class most exposed to the imbalance, is 0.867. The model made 6 false positives (real NORMAL predicted PNEUMONIA) and 5 false negatives (real PNEUMONIA predicted NORMAL), so it made slightly more false positives.

Class weighting ablation

In the notebook run, removing the class weights lowered NORMAL recall from 0.867 to 0.756, a drop of 0.111, but this is weak evidence because the effect did not repeat in a second run.

Model	NORMAL recall	False negatives	False positives
With class weights	0.867	5	6

Model	NORMAL recall	False negatives	False positives
Without class weights	0.756	1	11

The direction matches what theory predicts: without weights the model leans towards the majority PNEUMONIA class, so it raises more false alarms on NORMAL cases (11 against 6) and misses fewer true pneumonia cases (1 against 5). In the command-line run, however, NORMAL recall was identical with and without weights (0.822), with 6 and 4 false negatives respectively. With only 45 NORMAL test images, one image moves NORMAL recall by about 2 points, and the synthetic data is easy to separate, so the gain from weighting ranged from +0.00 to +0.11 between runs.

Whether weighting helps meaningfully on the real, harder dataset has not been measured. The `--ablation` option of the pipeline produces this comparison on real data.

Discussion

In a medical setting a false negative is usually the worse error. A missed pneumonia case goes untreated, while a false positive normally costs a follow-up check. Class weighting pushes the model towards recognising NORMAL cases, which trades some missed-disease risk for fewer false alarms, so both recalls must be read together: NORMAL recall shows whether the imbalance is handled, and PNEUMONIA recall shows how many sick patients are still caught. Which trade-off is acceptable is a clinical decision; the 0.5 decision threshold used here was not tuned.

What surprised me in preprocessing. The raw training images came in 99 different sizes and two colour modes, so nothing could even be batched before conversion. The order of steps also matters: computing class weights after augmentation, or augmenting the validation set, would make the reported numbers misleading. This section is a draft observation based on the synthetic data and should be replaced with your own after running the real dataset.

Limitations

- All results are from synthetic data, which is far easier to separate than real X-rays, so the accuracy here says nothing about real performance.
- The test set has only 45 NORMAL images, so single-run recall figures carry a margin of several points.
- Each configuration was trained once per run, with no repeated seeds or confidence intervals.

- The MobileNetV2 bonus is implemented but not verified: its ImageNet weights could not be downloaded in the build environment, so no transfer-learning comparison exists yet.
- The model is a learning exercise and must not be used for diagnosis.

Conclusion and reproduction

The pipeline works end to end: it inspects, splits, preprocesses, weights, trains and evaluates correctly, and its seven unit tests pass. Whether the approach performs well on real chest X-rays is still open until it is run on the Kaggle data.

To reproduce on the real dataset:

1. Install the requirements with `pip install -r requirements.txt -r requirements-dev.txt`.
2. Place your Kaggle API token in `~/.kaggle/kaggle.json`, then run `bash scripts/download_data.sh`.
3. Run `python run_pipeline.py --ablation`, adding `--mobilenet` for the transfer-learning bonus.
4. Read `results/writeup.md`, which is filled in with the real numbers, and update the tables in this report from it.
5. If you use the notebook instead, set `USE_SYNTHETIC = False` in its first code cell.